Model Library, entitlement catalogs, prices and deployments — internals

Data: `LibraryModels` is a snapshot of the gateway's `/v1/models` (extended attributes) written by `modelCostService.upsertLibrarySnapshot` on the Valkey `model-list-updated` event, the daily pull and the `refreshModelLibrary` action; rows are marked `absent`, never deleted. `ModelCatalogs` hold the default (exclusion-defined, never deletable), admin catalogs (`parent = null`) and user catalogs (`ownerEmail`, `parent` = the user's assigned catalog). Assignment lives on `Users.entitlementCatalog` (the admin cockpit chapter's Data Model section covers the `Users` entity): the `assignCatalog` and `unassignCatalog` bound actions write it, `getAssignedCatalog`, `affectedEmails` and the `libraryUsers` action read it. `ModelCatalogAssignments` is legacy — `usersService.migrateCatalogAssignments` drains it into `Users.entitlementCatalog` once at startup and it is never repopulated. Rules live in `services/admin/src/services/modelEntitlementService.ts`; handlers in `src/srv/admin-service-library.ts`.

Wire enforcement: `validation-service.ts` attaches `entitlement` (`{ mode: 'all', exclude? }` or `{ mode: 'list', include }`) to unified validations and to `credentialMetadata` on the SigV4 path. The gateway caches it with the credential (`unified-cache:*`) and filters `/v1/models` and `/openai/v1/models`, 404ing an unentitled id (`modelController.ts`; listing filtering is separate from the inference guard below). Inference is guarded by `enforceEntitlement(req, res, model)` (`utils/modelEntitlement.ts`), called after `substitute_models` resolution by the five controllers that carry it, on:

- `openaiController` — chat completions: `/openai/v1/chat/completions`, `/openai/api/v1/chat/completions` (two call sites: the `--deployed` branch and the plain one)
- `anthropicController` — messages: `/anthropic/v1/messages`, `/anthropic/v1/complete`, `/anthropic/v1/messages-beta`
- `responsesController` — `/openai/v1/responses`, `/openai/api/v1/responses` (two call sites: the requested model, then again after the `--deployed` sibling swap)
- `awsBedrockController` — `/aws-bedrock/model/:modelId/:subpath`
- `embeddingController` — `/openai/v1/embeddings`, `/openai/api/v1/embeddings`

A refusal is 403 `model_not_entitled` plus a security event; the call-site count per controller is pinned by `test/entitlement-guard-wiring.test.ts`. No block = unrestricted (standalone, local fallback). Cache invalidation on catalog/assignment changes goes through `cacheInvalidationService` (per user credentials via `bulkInvalidate`; a default-catalog change via `invalidatePattern('unified-cache:*')`, which deletes the ValKey keys AND publishes a `pattern_invalidation` the gateway subscriber hands to every registered cache's `clearByPattern` — a KEYS+DEL alone never reaches the gateway's in-process cache). `unified-cache:` is the ValKey key prefix only; the in-process map holds the same entries un-prefixed (`unified:apikey:<hash>`), so `UnifiedValidationCache.clearByPattern` also matches local keys against the pattern with that prefix stripped (`VALKEY_CACHE_KEY_PREFIX`, pinned by `services/gateway/test/unified-cache-clear-by-pattern.test.ts`). It also clears the ADMIN's own validation service instance cache (`apikey:*`, `unified_apikey:*`, `aws:*`, `unified_aws:*`). An administrator is never restricted: `entitlementFromRequest` (`services/gateway/src/utils/modelEntitlement.ts`) treats a caller whose `user` block carries the admin role as unrestricted before it looks at the assignment at all (spec §5's limitation, closed). The Users & Quotas app shows the user's assigned catalog read-only on its Entitlement section; the Entitlements & Quotas app's *Assignments* tab is still what writes it.

Prices: `ModelCosts.source` is `sap` or `manual`; `modelPriceService.setManualPrice` closes the current temporal row and inserts a manual one, which `updatePricingDatabase` never overwrites; `revertToSapPrice` restores the snapshot's SAP price. Display: `capacityUnitsPerMillion(cost, cuFactor)` = cost × 1000 × cuFactor, i.e. capacity units per 1M tokens from a per-1K cost, five decimals (SAP shows 1.50404 for 0.00079 at 1.90385). `sapCapacityService.computeSapNative`'s `genAiTokens` sum uses the same per-token rate math per usage event; a call carrying image or audio tokens replaces the plain `inputTokens × inputGenAiRate + outputTokens × outputGenAiRate` terms with `textIn × inputGenAiRate + audioIn × audioInputGenAiRate + textOut × outputGenAiRate + imageOut × imageOutputGenAiRate + audioOut × audioOutputGenAiRate`, where `audioIn = audioInputTokens`, `textIn = inputTokens − audioIn`, `imageOut = imageOutputTokens`, `audioOut =`

audioOutputTokens and textOut = outputTokens − imageOut − audioOut; imageOutputGenAiRate is ModelCosts.imageOutputCost / 1000 when maintained, else it falls back to the output rate (itself falling back to the input rate); audioInputGenAiRate/audioOutputGenAiRate are ModelCosts.audioInputCost / 1000 and audioOutputCost / 1000, falling back to the text rate of their direction. SQLite migration: docs/developer/sqlite-migrations/audio-tokens-sqlite-migration.sql.

Cache billing factors (providers.<key>.sap_cache_read_token_billing_factor / sap_cache_write_token_billing_factor, schema-allowed on anthropic only) scale the cache read and write tokens before their GenAI rate. sapCapacityService.cacheBillingFactorsByProvider() is the one place that decides which rows a factor applies to: a map keyed by the lower-cased provider key, a missing or negative value read as 1. A usage row's provider is the ModelCosts provider SAP publishes (Anthropic), so the match ignores case; a row with any other, unknown or missing provider gets 1. Live capture (computeSapNative → getCacheBillingFactors(provider)) and the daily cost recalculation (costRecalculationService.buildUpdateSQL, which binds the map as (CASE LOWER(<row>.provider) WHEN 'anthropic' THEN <f> ELSE 1 END) into both the Postgres and the SQLite statement, or 1 when no factor differs from 1) therefore give the same figures. Before this, live capture looked the key up case-sensitively and never applied a factor, while the recalculation applied the anthropic factor to every row it touched. The recalculation covers the last 30 days; on Postgres it rewrites only rows whose dollar costs drift more than 5% from ModelCosts, so rows captured before the fix with a factor other than 1 keep their old SAP-native figures (no forced recompute exists yet); SQLite reprices every active row in the window. Tests: test/integration/cost-recalculation-cache-factors.test.ts (both paths, one model per cache regime, unknown and missing provider), test/unit/services/cost-recalculation-factor-sql.test.ts (mixed-case key, the generated SQL).

Deployments: services/gateway/src/services/deploymentManagementService.ts ports the CLI's calls behind /api/admin/deployments (service key ADMIN_TO_GATEWAY, deployments:read/write). The route is mounted by mountDeploymentRoutes(app) and skipped entirely in standalone mode, where the service-key middleware would grant it to any caller without a credential. The admin's gatewayDeploymentClient.ts calls it; actions fetchDeployments, deploy, deploymentStatus on AdminService. A create clears the gateway model and discovery caches and emits deployment_created.

Snapshot rows are clamped to the CDS column widths (SNAPSHOT_WIDTHS in librarySnapshot.ts; the two JSON columns drop whole entries so they stay parseable), and a snapshot failure is logged and stepped over so it can never stop the pricing update. pruneChildren carries a visited set: the parent chain is data, and a cycle would otherwise never terminate — the ModelCatalogs UPDATE handler additionally refuses any parent that is not an admin catalog or the default.

Local SQLite dev: the running dev admin (cds serve) does not deploy schema changes, so an existing db/admin.db needs both steps by hand, with the admin stopped and after PRAGMA wal_checkpoint(TRUNCATE) and a copy of the file: ALTER TABLE sap_llm_gateway_admin_ModelCosts ADD COLUMN source NVARCHAR(8) DEFAULT 'sap', and the five new tables plus six new views taken from npx cds compile src/db src/srv --to sql --dialect sqlite (sap_llm_gateway_admin_LibraryModels, …ModelCatalogs, …ModelCatalogMembers, …ModelCatalogExclusions, …ModelCatalogAssignments; AdminService_LibraryModels, AdminService_ModelCatalogs, AdminService_ModelCatalogMembers, AdminService_ModelCatalogExclusions, AdminService_ModelCatalogAssignments, AdminService_ModelPrices) applied with sqlite3. Postgres (Docker/Kyma) gets all of it from schema_evolution: auto on deploy. The default catalog is seeded on the first entitlement lookup (ensureDefaultCatalog), and refreshModelLibrary fills the snapshot on demand.

Quota profiles: QuotaProfiles (db/schema/users.cds) is a named set of the same seven limits a Users row carries directly (requestsPerMinute, spendPerDay/Week/Month, tokensPerDay/Week/Month), unique on name (@assert.unique), with a virtual assignedUsers count filled in by the service's after-READ rather than modelled as a computed column of the projection — the same reason ApiKeys' field-control elements are virtual: a projection that adds one deploys as a view SQLite refuses to insert into. Users.quotaProfile (nullable) sits between a user's own limit fields and the platform: quotaLimits.effectiveLimits(user, profile, platform) resolves each of the seven fields independently — the user's own value first, then the assigned profile's, then platform.quotas, else unlimited — and returns a parallel LimitSource naming which of 'user' | 'profile' | 'platform' | 'unlimited' won for each field. defaultText() renders what an emptied field would fall back to ("1,000,000 (Standard profile)", "25.00 USD (platform)", "unlimited"), leaving the user's own value out of the resolution on purpose, so it keeps naming the fallback even while the user still carries a value of their own.

`userQuotaService.status/statusMany` resolve the assigned profile per user (`getProfile`/`getProfilesByIds` in `quotaProfilesService.ts`, batched once for a whole page) and hand `QuotaStatus.quotaProfileName`, `profileLimits` and `platformLimits` to callers, alongside the same `effectiveLimits` result the quota state document (`buildDocument`, published to Valkey) uses. `usersService.wireBlock` resolves the profile too, independently: it is where the gateway's requests-per-minute quota comes from on the synchronous validation path, not from the published document.

Write paths: admin CRUD on `QuotaProfiles` lives in `admin-service-quota-profiles.ts`. CREATE is redirected to the base table exactly like `ModelCatalogs` (a view SQLite refuses to INSERT into); `before` on CREATE/UPDATE trims and uniques the name and runs `validateConstraints` against the stored row overlaid by the patch (window ordering day <= week <= month, non-negative, integer where typed — the same rule a user's own constraints already obey). DELETE is refused with 409 `quota_profile_assigned` while any user still carries the profile (`assignedEmails`) — cascading would silently move them onto the platform defaults, which is left to an administrator's explicit choice. `Users.quotaProfile_ID` has exactly one write path: the bound actions `assignQuotaProfile`/`unassignQuotaProfile` (`setProfile`) — `admin-service-users.ts` deliberately keeps `quotaProfile_ID` out of `VIRTUAL_FIELDS`; the users-app annotates it `Common.FieldControl: #ReadOnly`, so cds drops the field from a draft PATCH payload before any handler runs, and the association can only move through the two actions. Every assignment, unassignment, profile edit and profile delete is audited (`recordAuditEvent`, actions `quota_profile.assign|unassign|update|delete`). An assignment change republishes that one user's quota document (`quota.publish`) and invalidates their credential caches (`invalidateForEmails(cds, [email], 'constraints')`); editing a profile's limits does the same for every user assigned to it, in one pass (`quota.publishMany` / `invalidateForEmails` over `assignedEmails`). `quotaProfileUsers()` is the bound function the model-library-app's profile mode reads for its Assignments table — one row per user (email, displayName, status, profileId, profileName), self-healing via `backfillUsers` like `libraryUsers`.

Seeding: three starters (Light / Standard / Power, spec 2026-09-08 §1) live in `quotaProfilesService.STARTER_PROFILES`; `ensureStarterProfiles(db)` inserts them only into an empty table (a `count(*)` check, not a name match), called from `initializeModelLibrary()` (`admin-service-library.ts`) right next to `ensureDefaultCatalog(db)` — the same startup hook, so a fresh or upgraded deployment gets both the default entitlement catalog and the three starter quota profiles on first boot, and never again once either table already holds a row.

Local SQLite dev: the running dev admin (`cds serve`) does not deploy schema changes here either, and `pnpm run db:migrate` (`cds deploy --to sqlite:db/admin.db`) recreates the database empty rather than migrating it — verified against a copy. An existing `db/admin.db` needs the same by-hand treatment as the catalogs above, with the admin stopped and after `PRAGMA wal_checkpoint(TRUNCATE)` and a copy of the file: the two `quotaProfile_ID` columns (`Users` and its draft table), the new `QuotaProfiles` table, and the `AdminService_Users` / new `AdminService_QuotaProfiles` views, taken from `npx cds compile src/db src/srv --to sql --dialect sqlite` and applied with `sqlite3`:

```
ALTER TABLE sap_llm_gateway_admin_Users ADD COLUMN quotaProfile_ID NVARCHAR(36);
ALTER TABLE AdminService_Users_drafts ADD COLUMN quotaProfile_ID NVARCHAR(36) NULL;

CREATE TABLE sap_llm_gateway_admin_QuotaProfiles (
  ID NVARCHAR(36) NOT NULL,
  createdAt TIMESTAMP_TEXT,
  createdBy NVARCHAR(255),
  modifiedAt TIMESTAMP_TEXT,
  modifiedBy NVARCHAR(255),
  name NVARCHAR(100) NOT NULL,
  description NVARCHAR(500),
  requestsPerMinute INTEGER,
  spendPerDay DECIMAL(12, 4),
  spendPerWeek DECIMAL(12, 4),
  spendPerMonth DECIMAL(12, 4),
  tokensPerDay BIGINT,
  tokensPerWeek BIGINT,
  tokensPerMonth BIGINT,
  PRIMARY KEY(ID),
  CONSTRAINT sap_llm_gateway_admin_QuotaProfiles_name UNIQUE (name)
);
```

```
DROP VIEW IF EXISTS AdminService_Users;
CREATE VIEW AdminService_Users AS SELECT
  Users_0.createdAt,
  Users_0.createdBy,
  Users_0.modifiedAt,
  Users_0.modifiedBy,
  Users_0.email,
  Users_0.displayName,
  Users_0.rolesSnapshot,
  Users_0.firstSeenAt,
  Users_0.lastSeenAt,
  Users_0.status,
  Users_0.statusChangedAt,
  Users_0.statusChangedBy,
  Users_0.statusReason,
  Users_0.requestsPerMinute,
  Users_0.spendPerDay,
  Users_0.spendPerWeek,
  Users_0.spendPerMonth,
  Users_0.tokensPerDay,
  Users_0.tokensPerWeek,
  Users_0.tokensPerMonth,
  Users_0.quotaResetAt,
  Users_0.entitlementCatalog_ID,
  Users_0.quotaProfile_ID
FROM sap_llm_gateway_admin_Users AS Users_0;

CREATE VIEW AdminService_QuotaProfiles AS SELECT
  QuotaProfiles_0.ID,
  QuotaProfiles_0.createdAt,
  QuotaProfiles_0.createdBy,
  QuotaProfiles_0.modifiedAt,
  QuotaProfiles_0.modifiedBy,
  QuotaProfiles_0.name,
  QuotaProfiles_0.description,
  QuotaProfiles_0.requestsPerMinute,
  QuotaProfiles_0.spendPerDay,
  QuotaProfiles_0.spendPerWeek,
  QuotaProfiles_0.spendPerMonth,
  QuotaProfiles_0.tokensPerDay,
  QuotaProfiles_0.tokensPerWeek,
  QuotaProfiles_0.tokensPerMonth
FROM sap_llm_gateway_admin_QuotaProfiles AS QuotaProfiles_0;
```

Postgres (Docker/Kyma) needs none of this — `schema_evolution: auto` adds it on deploy. The three starter profiles are then seeded on the next start, same as a fresh Postgres deployment (`ensureStarterProfiles`, called from the same startup hook as the default catalog).

Single SQLite connection: `@cap-js/sqlite` pools one connection, so a handler must never hold its request transaction across a gateway call — the gateway validates the service key by calling back into this admin's database. `getServiceApiKey` reads the key in a detached `cds.tx(fn)` for that reason; follow the same rule for any new admin→gateway call made from a request handler.